

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Харківський національний університет імені В.Н. Каразіна

Навчально-науковий інститут комп'ютерних наук та штучного інтелекту

Кафедра математичного моделювання та аналізу даних

ЛАБОЛАТОРНА РОБОТА № 5

на тему: «Робота з індексами та процедурами в MSSQL»

Виконав: студент 3 курсу групи кс31
Спеціальності 124 «Комп'ютерні
науки»

Кірієнко Данило

Прийняв: к.т.н., доцент каф. ММ та
АД

Коробчинський К.П.

Мета роботи:

Ознайомитися з поняттям індексів у системі керування базами даних Microsoft SQL Server, вивчити їх призначення, особливості використання та вплив на продуктивність запитів. Засвоїти оператори створення індексів, порядок їх застосування, а також методи реорганізації та видалення. Набути практичних навичок роботи з індексами шляхом створення, оптимізації та аналізу їх ефективності у базі даних.

Завдання 1.

Пройти тестування та додати скріншот з оцінкою у звіт.

Завдання 2.

За своїм варіантом (основного, по якому були виконані практичні роботи 3, 4, 5 та 6) додати опис предметної області (скопійувати з варіанту), логічну і фізичну моделі (скопійувати з практичної роботи №5). Наступні запити виконати в зручному для Вас середовищі: Docker, Віртуальна машина з MSSQL, файлова БД або встановлений на локальному комп'ютері екземпляр MSSQL.

Завдання 3.

Додати наступні індекси у таблиці Вашого варіанта

- о Некластеризований індекс;
- о Кластеризований індекс;
- о Унікальний індекс;
- о Індекс з включеними стовпцями;
- о Фільтрований індекс;

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Таблицю з використанням й без використання індексів;
- Зберегти код запитів додавши у файл QUERY.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 4.

Проаналізуйте Ваші таблиці з даними 500, 1000, 10 000 рядків, відповідно наповнивши ці таблиці. Для аналізу запитів використовуйте запити з свого варіанта, не менше 3 будь-яких запитів на Ваш вибір. Результат додайте у звіт у вигляді зведеної таблиці. Та проаналізуйте типи індексів та їх використання у залежності від кількості рядків.

Завдання 5.

Проведіть аудит усіх індексів у базі даних: визначте кількість, тип, унікальність та фрагментацію кожного. Підготуйте звіт у табличній формі.

- ім'я індексу;
- тип індексу (кластеризований, некластеризований тощо);
- унікальність;
- рівень фрагментації;
- таблицю, до якої належить індекс;
- кількість сторінок, які використовує індекс.

Завдання 6.

Написати скрипт T-SQL який буде використовувати Системні збережені процедури. Зробити оброку з 3-а процедурами, додати специфікацію їх, та опишіть як саме можна їх використовувати у Вашій БД.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 7.

Написати скрипт T-SQL який буде використовувати Глобальні тимчасові збережені. Зробити оброку з 3-а процедурами, додати специфікацію їх, та опишіть як саме можна їх використовувати у Вашій БД.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 8.

Написати скрипт T-SQL який буде використовувати Тимчасові збережені процедури. Зробити обробку з 3-а процедурами, додати специфікацію їх, та опишіть як саме можна їх використовувати у Вашій БД.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 9.

Написати скрипт T-SQL який буде використовувати Збережені процедури користувачів. Виконати завдання (з частини Транзакції), додати специфікацію їх, та опишіть їх.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 10.

Написати скрипт T-SQL який додасть за зазначенням параметра відповідну кількість рядків у таблицю.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 11.

Для таблиці, що має первинний ключ, що генерується послідовністю, створити функцію, яка як параметри приймає значення полів запису, крім первинного ключа. Ця процедура повинна отримувати з послідовності новий первинний ключ і вставити новий запис із цим ключем та із зазначеними у вигляді параметрів значеннями полів. В результаті процедура повинна повертати первинний ключ вставленого запису або null, якщо запис не було вставлено.

- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл PROCEDURES.SQL та оновити його на своїй сторінці на GitHub.

Завдання 12.

Завантажити файли до Git-репозиторію та додати посилання у звіт. Репозиторій повинен бути закритим, а доступ відкритий лише для викладачів практики, лабораторних робіт та лектора (k.korobchinskiy@karazin.ua). В репозиторії створити або оновити README.MD файл додавши використання процедур з коротким поясненням зроблених завдань.

Використовуючи мову T-SQL зробити скрипти.

- створення таблиць та змінення структури таблиць БД – SETUP.SQL;
- занесення інформації в таблиці БД – INSERT.SQL.
- зміни/оновлення/видалення інформації у таблицях БД – UPDATE.SQL;
- запитів – QUERY.SQL.
- функцій – FUNCTIONS.SQL.

Всі файли за архівувати та прикипіти до STM.

Завдання 13.

Додати висновки. До оформити звіт та завантажити на STM.

Хід роботи:

Завдання 2.

Вариант №15

Ферма

На ферме имеется несколько участков земли, им присвоены уникальные номера. Каждый участок характеризуется площадью, наличием или отсутствием орошения, посеянной в текущем сезоне культурой. Известна средняя урожайность каждой из возделываемых культур, а также перечень внесенных на каждый участок в данном сезоне удобрений. Для каждой культуры известно: название, урожайность по участкам, нужно ли для нее орошение, какие нужны удобрения. Каждый участок обслуживает одна бригада, но любая бригада может обслуживать более одного участка. Бригада имеет уникальный номер и характеризуется: Ф.И.О. бригадира, Ф.И.О. работников, количеством единиц каждого вида сельхозтехники. О каждом работнике известно, какими видами сельхозмашин (одним, несколькими, ни одним) он может управлять. Работники трудятся на любом из участков бригады, пользуясь техникой, которая есть в бригаде.

Запросы

- ✓ Определить площадь, которая отведена на ферме под каждую культуру с указанием, на каком количестве участков посеяна каждая культура;
- ✓ Вывести всех рабочих заданной бригады с перечнем сельхозтехники, на которой они умеют работать, если такая техника в бригаде имеется, а также определить те виды техники, на которой могут работать члены бригады, но в бригаде такой техники нет, хотя она есть в других бригадах
- для участков данной бригады найти культуру с максимальной урожайностью и проверить, посеяна ли эта культура в текущем сезоне на том участке, где ее урожайность максимальна
- ✓ Найти бригады, которые располагают заданным видом техники в количестве, большем чем среднее количество этой техники по бригадам на всей ферме;
- ✓ Составить список участков (с фамилиями их бригадиров), на которых внесены удобрения, не соответствующие культуре, или не выполняются требования по орошению культуры.

Транзакции

- ✓ Купить новую сельхозтехнику
- ✓ Нанять/уволить сельхозработчик (в том числе и бригадиров).

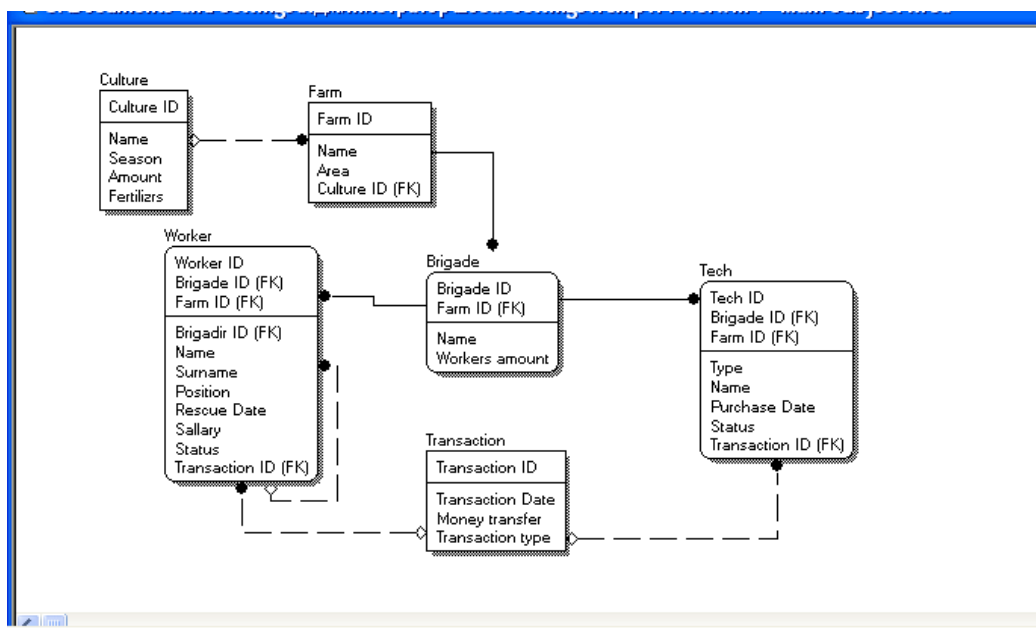


Рисунок 1 - Логічна модель

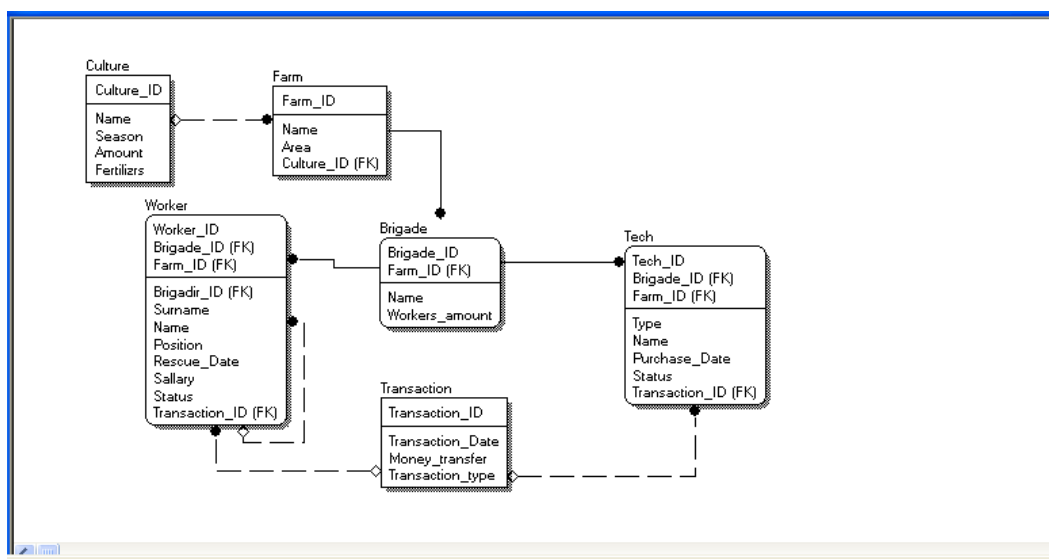


Рисунок 2 - Фізична модель

Завдання 3.

Створемо відповідні індекси:

```

CREATE NONCLUSTERED INDEX IX_Worker_Position
ON Worker (Position);
GO

CREATE UNIQUE INDEX IX_Transaction_TypeAmount
ON [Transaction] (Transaction_type, Money_transfer);
GO
  
```

```

CREATE NONCLUSTERED INDEX IX_Worker_Salary
ON Worker (Salary)
INCLUDE (Name, Surname);
GO

CREATE NONCLUSTERED INDEX IX_Worker_ActiveStatus
ON Worker (Status)
WHERE Status = 'Active';
GO

```

Також намагався створити кластерний індекс, але виявилось що первинний ключ і так є кластерним індексом сам по собі, тож кожна таблиця вже його має.

Завдання 4.

Для легкого наповнення таблиці worker даними такою великою кількістю рядків, попросив чат гпт згенерувати відповідний скрипт:

```

DECLARE @CurrentMax INT = (SELECT MAX(Worker_ID) FROM Worker);
DECLARE @StartID INT =
    CASE
        WHEN @CurrentMax IS NULL THEN 100000
        WHEN @CurrentMax < 100000 THEN 100000
        ELSE @CurrentMax + 1
    END;

SELECT DISTINCT Position INTO #Positions FROM Worker WHERE Position IS NOT NULL;
SELECT DISTINCT Status INTO #Statuses FROM Worker WHERE Status IS NOT NULL;
SELECT DISTINCT Tech_Type INTO #Types FROM Worker WHERE Tech_Type IS NOT NULL;

WITH Brigades AS (
    SELECT Brigade_ID, Farm_ID FROM Brigade
),
Numbers AS (
    SELECT TOP (100) ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS N FROM sys.all_objects
),
AllRows AS (
    SELECT
        B.Brigade_ID,

```

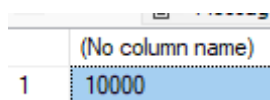
```

        B.Farm_ID,
        B.Brigade_ID AS Brigadir_ID,
        ROW_NUMBER() OVER (ORDER BY B.Brigade_ID, N.N) AS RowNum
    FROM Brigades B
    CROSS JOIN Numbers N
)
INSERT INTO Worker (
    Worker_ID, Brigade_ID, Farm_ID, Brigadir_ID,
    Name, Surname, Position, Rescue_Date, Salary,
    Status, Transaction_ID, Tech_Type
)
SELECT
    @StartID + RowNum,
    Brigade_ID,
    Farm_ID,
    Brigadir_ID,
    CONCAT('Name', @StartID + RowNum),
    CONCAT('Surname', @StartID + RowNum),
    (SELECT TOP 1 Position FROM #Positions ORDER BY NEWID()),
    DATEADD(DAY, -ABS(CHECKSUM(NEWID())) % 1800, GETDATE()),
    5000 + ABS(CHECKSUM(NEWID())) % 15000,
    (SELECT TOP 1 Status FROM #Statuses ORDER BY NEWID()),
    NULL,
    (SELECT TOP 1 Tech_Type FROM #Types ORDER BY NEWID())
FROM AllRows
WHERE @StartID + RowNum <> Brigade_ID;

DROP TABLE #Positions;
DROP TABLE #Statuses;
DROP TABLE #Types;

```

Виконуємо простий запит для перевірки `SELECT COUNT(*) FROM Worker WHERE Worker_ID >= 100000`; , має вийти 10к (загало ще є початкова сотня + 2 робіників яких ми додавали в минулій лабораторній):



	(No column name)
1	10000

Рисунок 3 - кількість вставлених робітників

Далі виконуємо наші запити з варіанта, обмеживши кількість рядків:

```

SELECT TOP (500) * FROM dbo.fn_CultureAreaStats();
SELECT TOP (1000) * FROM dbo.fn_CultureAreaStats();
SELECT TOP (10000) * FROM dbo.fn_CultureAreaStats();

```

```

SELECT TOP (500) * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');
SELECT TOP (1000) * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');
SELECT TOP (10000) * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');

SELECT TOP (500) * FROM dbo.fn_FarmFertilizerCheck();
SELECT TOP (1000) * FROM dbo.fn_FarmFertilizerCheck();
SELECT TOP (10000) * FROM dbo.fn_FarmFertilizerCheck();

```

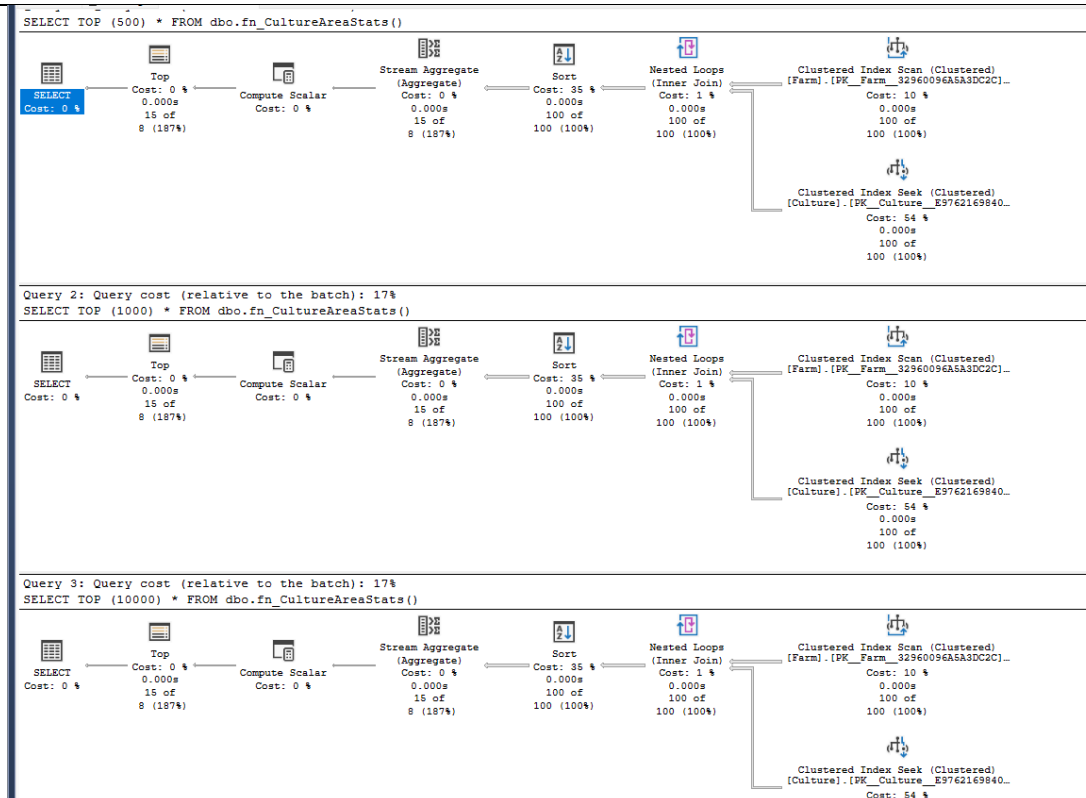


Рисунок 4 - аналіз першого запиту з 500,1000,10000

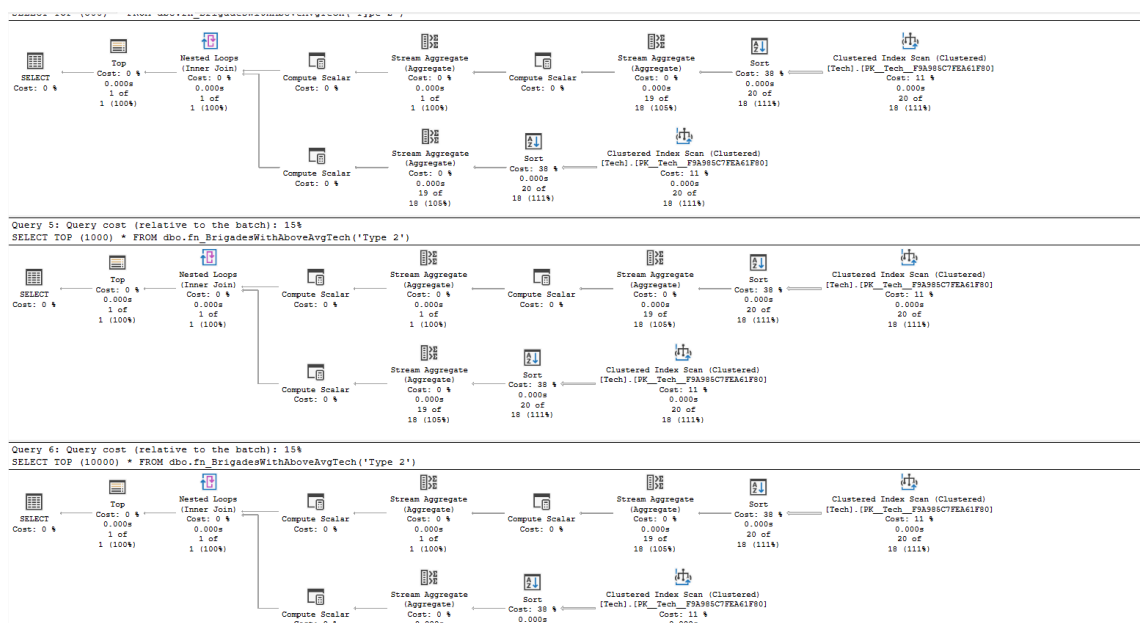


Рисунок 5 - аналіз 3 запиту з 500,1000,10000

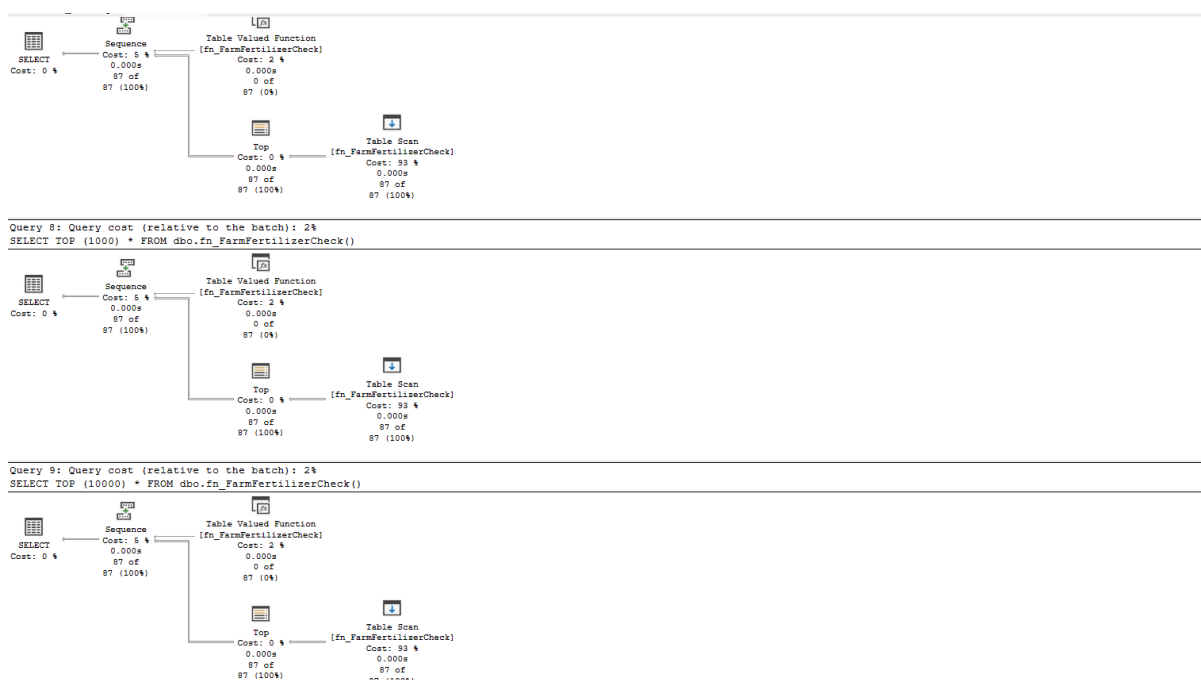


Рисунок 6 - аналіз запису 4 з 500,1000,10000

З аналізу видно, що кластерні індекси і так використовуються, але є сенс в потрібних місцях спробувати додати нонкластерні:

```
CREATE NONCLUSTERED INDEX IX_Farm_Culture_Area
ON Farm (Culture_ID, Area);

CREATE NONCLUSTERED INDEX IX_Tech_Type_BrigadeFarm
ON Tech (Type, Brigade_ID, Farm_ID);

CREATE NONCLUSTERED INDEX IX_Culture_Name_Fertilizers
ON Culture (Name, Fertilizers);

CREATE NONCLUSTERED INDEX IX_Farm_CultureID
ON Farm (Culture_ID, Farm_ID);

CREATE NONCLUSTERED INDEX IX_Worker_Join
ON Worker (Worker_ID, Brigade_ID, Farm_ID);
```

Тепер спробуємо знову виконати наші запити, цього разу вони мають виконатися ефективніше. В таблицю закину вже оптимізовані результати:

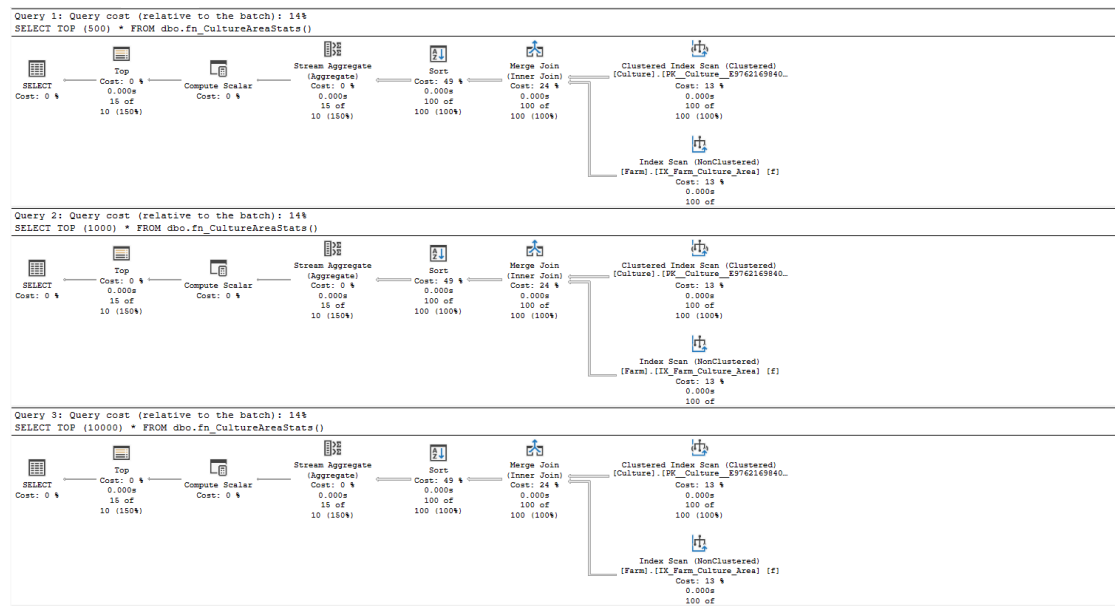


Рисунок 7 - аналіз оптимізованого запиту 1

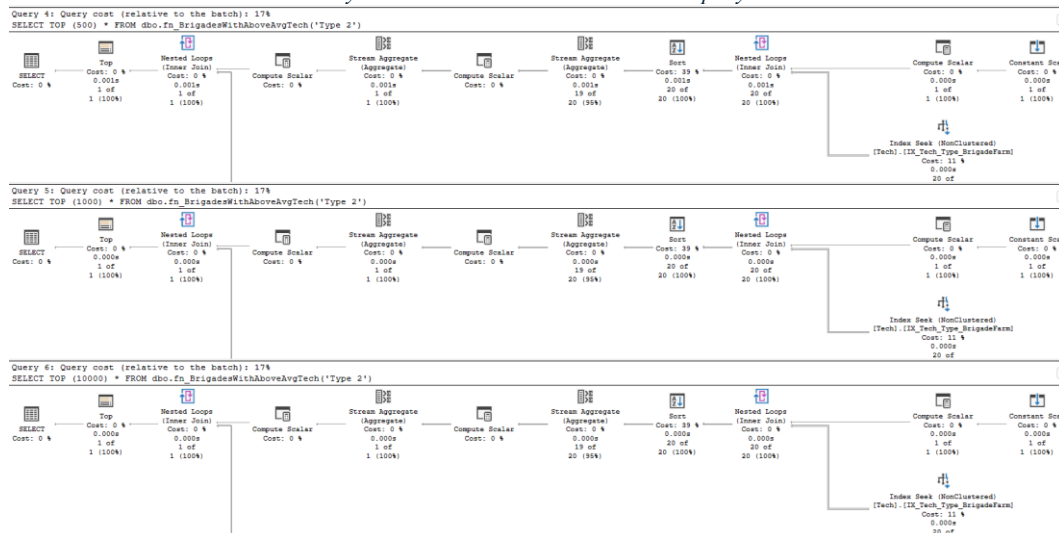


Рисунок 8 - аналіз оптимізації запиту 3

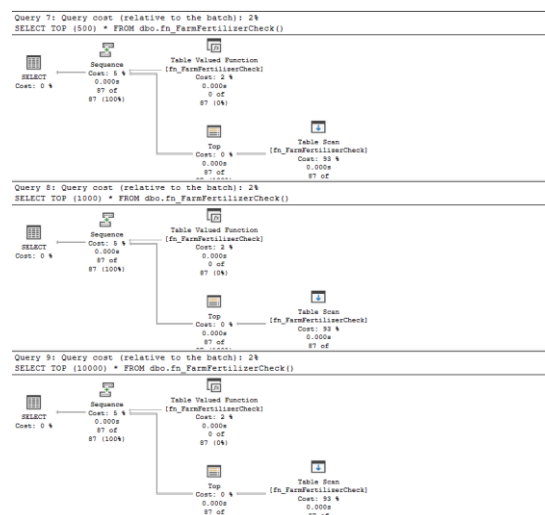


Рисунок 9 - аналіз оптимізації запиту 4

№	Функція	Рядків	Тип індексу	Використання індексу	Тип доступу	Коментар
1	fn_CultureAreaStats()	500	NONCLUSTERED (Culture_ID, Area) (Farm)	Так	Index Scan	План перейшов на Merge Join, повністю використовується індекс
2	fn_CultureAreaStats()	1000	NONCLUSTERED (Culture_ID, Area) (Farm)	Так	Index Scan	Повна відповідність по плану, агрегація через Stream Aggregate
3	fn_CultureAreaStats()	10000	NONCLUSTERED (Culture_ID, Area) (Farm)	Так	Index Scan	Ефективний при великих об'ємах, сортування оптимізоване
4	fn_BrigadesWithAboveAvgTech('Type')	500	NONCLUSTERED (Type, Brigade_ID, Farm_ID) (Tech)	Так	Index Seek	Використовується Seek замість повного сканування
5	fn_BrigadesWithAboveAvgTech('Type')	1000	NONCLUSTERED (Type, Brigade_ID, Farm_ID) (Tech)	Так	Index Seek	Ефективна агрегація по Type + Group By
6	fn_BrigadesWithAboveAvgTech('Type')	10000	NONCLUSTERED (Type, Brigade_ID, Farm_ID) (Tech)	Так	Index Seek	Підвищення швидкодії при великій вибірці
7	fn_FarmFertilizerCheck()	500	Culture(Name, Fertilizers)	Не видно напряму	Table Valued Function	не розкриває внутрішню логіку
8	fn_FarmFertilizerCheck()	1000	Culture(Name, Fertilizers)	Не видно напряму	Table Valued Function	не розкриває внутрішню логіку
9	fn_FarmFertilizerCheck()	10000	Culture(Name, Fertilizers)	Не видно напряму	Table Scan	не розкриває внутрішню логіку

Завдання 5.

```

SELECT
    i.name AS IndexName,
    i.type_desc AS IndexType,
    i.is_unique AS IsUnique,
    ps.avg_fragmentation_in_percent AS FragmentationPercent,
    t.name AS TableName,
    ps.page_count AS PageCount
FROM sys.indexes i
JOIN sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')
ps
    ON i.object_id = ps.object_id AND i.index_id = ps.index_id
JOIN sys.tables t
    ON i.object_id = t.object_id
WHERE i.name IS NOT NULL
ORDER BY FragmentationPercent DESC;

```

Експортую таблицю як пдф і потім переджу її в ворд

	IndexName	IndexType	IsUnique	FragmentationPerce	TableName	
1	IX_Worker_Salary	NONCLUSTERED	0	97,0588235294118	Worker	102
2	IX_Worker_Position	NONCLUSTERED	0	5,40540540540541	Worker	37
3	IX_Worker_Join	NONCLUSTERED	0	4,34782608695652	Worker	23
4	UQ_WorkerID	NONCLUSTERED	1	4,34782608695652	Worker	23
5	PK__Worker__F3FFD610EA496765	CLUSTERED	1	3,10077519379845	Worker	129
6	IX_Worker_ActiveStatus	NONCLUSTERED	0	2,85714285714286	Worker	35
7	PK__Tech__F9A985C7FEA61F80	CLUSTERED	1	0	Tech	1
8	IX_Tech_Type_BrigadeFarm	NONCLUSTERED	0	0	Tech	1
9	PK__Culture__E9762169840D8363	CLUSTERED	1	0	Culture	1
10	IX_Culture_Name_Fertilizers	NONCLUSTERED	0	0	Culture	1
11	PK__Farm__32960096A5A3DC2C	CLUSTERED	1	0	Farm	1
12	IX_Farm_Culture_Area	NONCLUSTERED	0	0	Farm	1
13	IX_Farm_CultureID	NONCLUSTERED	0	0	Farm	1
14	PK__Brigade__0A149E4103744872	CLUSTERED	1	0	Brigade	1
15	PK__Transact__9A8D5625007EBA...	CLUSTERED	1	0	Transaction	1
16	IX_Transaction_TypeAmount	NONCLUSTERED	1	0	Transaction	1

Завдання 6.

`sp_help 'Worker'` - Дозволяє подивитися всі колонки, типи даних, ключі та індекси таблиці Worker. Зручно при перевірці структури перед побудовою запитів або під час аудиту БД.

`sp_spaceused 'Farm'` - Показує скільки місця займає таблиця Farm — корисно при аналізі навантаження, особливо для оптимізації зберігання даних.

`sp_helpindex 'Culture'` - Дає перелік всіх індексів, їх тип, поля, унікальність.

```
EXEC sp_help 'Worker';
EXEC sp_spaceused 'Farm';
EXEC sp_helpindex 'Culture';
```

Results Messages										
	Name	Owner	Type	Created_datetime						
1	Worker	dbo	user table	2025-05-28 18:04:41.790						
	Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation
4	Brigadir_ID	int	no	4	10	0	yes	(n/a)	(n/a)	NULL
5	Name	varchar	no	50			yes	no	yes	Latin1_General_CI_AS
6	Surname	varchar	no	50			yes	no	yes	Latin1_General_CI_AS
7	Position	varchar	no	50			yes	no	yes	Latin1_General_CI_AS
8	Rescue_Date	date	no	3	10	0	yes	(n/a)	(n/a)	NULL
9	Salary	decimal	no	9	10	2	yes	(n/a)	(n/a)	NULL
10	Status	varchar	no	50			yes	no	yes	Latin1_General_CI_AS
11	Transaction_ID	int	no	4	10	0	yes	(n/a)	(n/a)	NULL
	Identity	Seed	Increment	Not For Replication						
1	No identity column defined.	NULL	NULL	NULL						
	RowGuidCol									
1	No rowguidcol column defined.									
	Data_located_on_filegroup									
1	PRIMARY									
	index_name	index_description		index_keys						
1	IX_Worker_ActiveStatus	nonclustered located on PRIMARY		Status						
2	IX_Worker_Join	nonclustered located on PRIMARY		Worker_ID, Brigade_ID, Fam_ID						
3	IX_Worker_Position	nonclustered located on PRIMARY		Position						
4	IX_Worker_Salary	nonclustered located on PRIMARY		Salary						
5	PK_Worker_F3FFD...	clustered, unique, primary key loc...		Worker_ID, Brigade_ID, Fam_ID						
6	UQ_WorkerID	nonclustered, unique, unique key...		Worker_ID						
	constraint_type	constraint_name	delete_action	update_action	status_enabled	status_for_replication	constraint_keys			
1	FOREIGN KEY	FK_Worker_412EB0B6	No Action	No Action	Enabled	Is_For_Replication	Brigade_ID, Fam_ID			
2							REFERENCES fam_db.dbo.Brigade (Brigade_ID, Fam...			
3	FOREIGN KEY	FK_Worker__Transact...	No Action	No Action	Enabled	Is_For_Replication	Transaction_ID			
4							REFERENCES fam_db.dbo.Transaction (Transaction_...			
5	FOREIGN KEY	FK_Brigadir	No Action	No Action	Enabled	Is_For_Replication	Brigadir_ID			
6							REFERENCES fam_db.dbo.Worker (Worker_ID)			
7	PRIMARY K...	PK_Worker_F3FFD6...	(n/a)	(n/a)	(n/a)	(n/a)	Worker_ID, Brigade_ID, Fam_ID			
8	UNIQUE (no...	UQ_WorkerID	(n/a)	(n/a)	(n/a)	(n/a)	Worker_ID			
	Table is referenced by foreign key									
1	fam_db.dbo.Worker: FK_Brigadir									

Рисунок 10 - вивід сп хелп по таблиці воркер

	name	rows	reserved	data	index_size	unused
1	Fam	100	216 KB	8 KB	40 KB	168 KB

Рисунок 11 - місце займане таблицею фарм

	index_name	index_description	index_keys
1	IX_Culture_Name_Fertilizers	nonclustered located on PRIMARY	Name, Fertilizers
2	PK_Culture_E9762169840D8363	clustered, unique, primary key located on PRIMARY	Culture_ID

Рисунок 12 - перелік індексів культур

Завдання 8.

Тимчасові збережені процедури відповідно з назви не зберігаються а базі даних назавжди і існують тільки протягом поточної сесії.

```
CREATE PROCEDURE #TempWorkerCount
AS
BEGIN
    SELECT COUNT(*) AS WorkerCount FROM Worker;
```

```

END;
GO

CREATE PROCEDURE #TempFarmSummary
AS
BEGIN
    SELECT Culture_ID, COUNT(*) AS FarmsCount, SUM(Area) AS TotalArea
    FROM Farm
    GROUP BY Culture_ID;
END;
GO

CREATE PROCEDURE #TempTopBrigades
AS
BEGIN
    SELECT TOP 5 Brigade_ID, COUNT(*) AS TechCount
    FROM Tech
    GROUP BY Brigade_ID
    ORDER BY TechCount DESC;
END;
GO

EXEC #TempWorkerCount;
EXEC #TempFarmSummary;
EXEC #TempTopBrigades;

```

	WorkerCount
1	10102

	Culture_ID	FarmsCount	TotalArea
1	2	1	71
2	4	4	603
3	5	2	286
4	6	1	151
5	7	1	67
6	9	2	262
7	10	1	173
8	11	1	139
9	12	1	52

	Brigade_ID	TechCount
1	52	3
2	6	1
3	5	1
4	4	1
5	3	1

Рисунок 13 - результат виводу тимчасових процедур

Завдання 9.

```

CREATE PROCEDURE AddNewWorker
    @Worker_ID INT,
    @Name NVARCHAR(50),
    @Surname NVARCHAR(50),
    @Farm_ID INT,
    @Brigade_ID INT
AS
BEGIN
    BEGIN TRANSACTION;

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Farm WHERE Farm_ID = @Farm_ID)
        BEGIN
            RAISERROR('Farm not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM Brigade WHERE Brigade_ID = @Brigade_ID
AND Farm_ID = @Farm_ID)
        BEGIN
            RAISERROR('Brigade not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        INSERT INTO Worker(Worker_ID, Name, Surname, Farm_ID, Brigade_ID)
VALUES (@Worker_ID, @Name, @Surname, @Farm_ID, @Brigade_ID);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE UpdateFertilizerForCulture
    @Culture_ID INT,
    @NewFertilizer NVARCHAR(50)
AS
BEGIN
    BEGIN TRANSACTION;
    BEGIN TRY
        UPDATE Culture
        SET Fertilizers = @NewFertilizer
        WHERE Culture_ID = @Culture_ID;

        COMMIT TRANSACTION;
    
```

```

    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE RemoveTechByID
    @Tech_ID INT
AS
BEGIN
    BEGIN TRANSACTION;
    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Tech WHERE Tech_ID = @Tech_ID)
        BEGIN
            RAISERROR('Tech not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        DELETE FROM Tech WHERE Tech_ID = @Tech_ID;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

```

1. Процедура додавання нового працівника з перевіркою та транзакцією
2. Процедура оновлення добрив для культури
3. Процедура видалення техніки з перевіркою

Додаємо:

```

EXEC AddNewWorker
    @Worker_ID = 157,
    @Name = 'Ivan',
    @Surname = 'petr',
    @Farm_ID = 1,
    @Brigade_ID = 1;

EXEC UpdateFertilizerForCulture
    @Culture_ID = 1,
    @NewFertilizer = 'Organic';

```

```
EXEC RemoveTechByID @Tech_ID = 3;
```

І за допомогою

```
SELECT * FROM Worker WHERE Name = 'Ivan' AND Surname = 'petr';
```

```
SELECT * FROM Culture WHERE Culture_ID = 1;
```

```
SELECT * FROM Tech WHERE Tech_ID = 3;
```

перевіряємо результат:

	Worker_ID	Brigade_ID	Fam_ID	Brigadir_ID	Name	Surname	Position	Rescue_Date	Salary	Status	Transaction_ID	Tech_Type
1	157	1	1	NULL	Ivan	petr	NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 14 - доданий працівник

	Culture_ID	Name	Season	Amount	Fertilizers
1	1	Wheat	Summer	34	Organic

Рисунок 15 - нове добриво

	Tech_ID	Brigade_ID	Fam_ID	Name	Type	Purchase_Date	Status	Transaction_ID
--	---------	------------	--------	------	------	---------------	--------	----------------

Рисунок 16 - прибрали всю техніку типу 2

Завдання 10.

```
CREATE PROCEDURE InsertTestTech
    @Count INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @CurrentMaxID INT = ISNULL((SELECT MAX(Tech_ID) FROM Tech),
    100000);

    ;WITH Numbers AS (
        SELECT TOP (@Count) ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS N
        FROM sys.all_objects
    ),
    RandomBrigades AS (
        SELECT
            ROW_NUMBER() OVER (ORDER BY NEWID()) AS RowNum,
            Brigade_ID,
            Farm_ID
        FROM Brigade
    )
```

```

INSERT INTO Tech (Tech_ID, Name, Farm_ID, Brigade_ID)
SELECT
    @CurrentMaxID + n.N,
    CONCAT('Tech', @CurrentMaxID + n.N),
    b.Farm_ID,
    b.Brigade_ID
FROM Numbers n
CROSS APPLY (
    SELECT TOP 1 Brigade_ID, Farm_ID
    FROM RandomBrigades
    WHERE RowNum = ((ABS(CHECKSUM(NEWID()), n.N)) % (SELECT COUNT(*)
FROM Brigade)) + 1)
) b;
END;
GO

```

Відповідно спробуємо вставити 50 одиниць за допомогою EXEC InsertTestTech @Count = 50; і перевіримо за допомогою SELECT * FROM Tech ORDER BY Tech_ID DESC;:

	Tech_ID	Brigade_ID	Farm_ID	Name	Type	Purchase_Date	Status	Transaction_ID
1	2099	77	77	Tech2099	NULL	NULL	NULL	NULL
2	2098	84	84	Tech2098	NULL	NULL	NULL	NULL
3	2097	67	67	Tech2097	NULL	NULL	NULL	NULL
4	2092	25	25	Tech2092	NULL	NULL	NULL	NULL
5	2091	79	79	Tech2091	NULL	NULL	NULL	NULL
6	2090	63	63	Tech2090	NULL	NULL	NULL	NULL
7	2089	70	70	Tech2089	NULL	NULL	NULL	NULL
8	2087	58	58	Tech2087	NULL	NULL	NULL	NULL
9	2086	6	6	Tech2086	NULL	NULL	NULL	NULL
10	2084	79	79	Tech2084	NULL	NULL	NULL	NULL
11	2083	100	100	Tech2083	NULL	NULL	NULL	NULL
12	2082	4	4	Tech2082	NULL	NULL	NULL	NULL
13	2081	57	57	Tech2081	NULL	NULL	NULL	NULL
14	2080	28	28	Tech2080	NULL	NULL	NULL	NULL
15	2078	86	86	Tech2078	NULL	NULL	NULL	NULL
16	2077	82	82	Tech2077	NULL	NULL	NULL	NULL
17	2076	61	61	Tech2076	NULL	NULL	NULL	NULL
18	2075	53	53	Tech2075	NULL	NULL	NULL	NULL
19	2072	31	31	Tech2072	NULL	NULL	NULL	NULL
20	2069	26	26	Tech2069	NULL	NULL	NULL	NULL
21	2068	70	70	Tech2068	NULL	NULL	NULL	NULL
22	2066	57	57	Tech2066	NULL	NULL	NULL	NULL
23	2065	47	47	Tech2065	NULL	NULL	NULL	NULL
24	2064	37	37	Tech2064	NULL	NULL	NULL	NULL
25	2063	96	96	Tech2063	NULL	NULL	NULL	NULL
26	2062	32	32	Tech2062	NULL	NULL	NULL	NULL
27	2060	40	40	Tech2060	NULL	NULL	NULL	NULL
28	2059	6	6	Tech2059	NULL	NULL	NULL	NULL
29	2058	28	28	Tech2058	NULL	NULL	NULL	NULL
30	2057	59	59	Tech2057	NULL	NULL	NULL	NULL
31	2056	63	63	Tech2056	NULL	NULL	NULL	NULL
32	2055	1	1	Tech2055	NULL	NULL	NULL	NULL
33	2052	50	50	Tech2052	NULL	NULL	NULL	NULL
34	2049	31	31	Tech2049	NULL	NULL	NULL	NULL
35	2048	100	100	Tech2048	NULL	NULL	NULL	NULL
36	2045	78	78	Tech2045	NULL	NULL	NULL	NULL
37	2040	25	25	Tech2040	NULL	NULL	NULL	NULL
38	2038	76	76	Tech2038	NULL	NULL	NULL	NULL
39	2037	24	24	Tech2037	NULL	NULL	NULL	NULL

Рисунок 17 - створена техніка

Завдання 11.

Незважаючи на те, що завдання просить функцію, процедура ніби за темою більше підходить, та й по типу здається що звучніше.

```
CREATE PROCEDURE InsertTechWithSequenceProc
    @Name NVARCHAR(100),
    @Farm_ID INT,
    @Brigade_ID INT,
    @Type NVARCHAR(50),
    @NewTechID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        SET @NewTechID = NEXT VALUE FOR Seq_Tech_ID;

        INSERT INTO Tech (Tech_ID, Name, Farm_ID, Brigade_ID, Type)
        VALUES (@NewTechID, @Name, @Farm_ID, @Brigade_ID, @Type);
    END TRY
    BEGIN CATCH
        SET @NewTechID = NULL;
    END CATCH
END;
GO
```

	InsertedID									
1	1002	65	1002	1	1	New1	NULL	NULL	NULL	NULL
		66	1001	1	1	New1	NULL	NULL	NULL	NULL

Рисунок 18 - техніка додалася, виводить айди

Висновок:

У ході виконання лабораторної роботи було розроблено та протестовано збережені процедури різних типів у середовищі SQL Server. Було реалізовано системні, тимчасові та користувацькі збережені процедури, що продемонстрували можливість автоматизації обробки даних у базі. Зокрема, створені процедури забезпечують валідацію, використання транзакцій, вставку даних із використанням послідовності, а також масове додавання записів із контрольованими зв'язками між таблицями. Таким чином, отримано практичні навички створення ефективних SQL-скриптів і покращено розуміння механізмів роботи серверних процедур.

GitHub з файлами практичної: <https://github.com/ddanq4/3-obd/tree/main/2%20seministr/lab5>